LAB ACTION PLAN FOR WEEK 6

Objectives:

Students will able to:

- Learn how to define and run multiple interdependent services (e.g., web server, database) in a single configuration file.
- Gain skills in writing and interpreting docker-compose.yml files for service setup.
- Deploy the same setup across different machines without manual configuration.
- Configure container networking and persistent storage within Compose.
- Reduce setup time and enable faster iteration during application development.

Students will performHands-on running multi container applicationsusing docker composeand make document that include the execution of each step along withscenario-based question solutions after completing the tasks.

What is Docker Compose?

Docker Compose is a tool used to define and run multi-container Docker applications. It allows you to define services, networks, and volumes that your application needs, all in a single file. This makes it easier to manage complex applications that require multiple containers (e.g., a web server and a database).

Structure of a Docker Compose File

A Docker Compose file is written in YAML (YAML Ain't Markup Language), a human-readable data format used for configuration. In the Docker context, it helps define the services, networks, and volumes needed for an application.

Basic Components of a Docker Compose YAML File

1. **Version**: Specifies which version of the Docker Compose file format you're using. Each version comes with its own set of features.

   version:'3.8'

2. **Services**: This section defines all the containers you want to run for your application. Each service represents a container (like a web server or database).

   Each service includes:

- **Image**: The Docker image to use.
- **Ports**: Ports to map between the container and your host machine.
- **Environment**: Environment variables required by the service.
- **Dependencies**: Which other services a container depends on.

Example:

```
services:
web:
image:nginx
ports:
-"8060:80"
db:
image:tomee
ports:
-"8050:8080"
```

3. **Networks**: Define networks to allow different services to communicate with each other. Docker Compose automatically creates a default network if not specified.

4. **Volumes**: Used for persistent storage, allowing data to persist even after the container stops or is removed.

Example Docker Compose File (Simple)

Here's a basic example of a Docker Compose file that runs WordPress and MySQL together:

```
version: '3.8'  # Docker Compose file format version

services:
wordpress:  # WordPress service
   image: wordpress:latest
   ports:
    - "8080:80"  # Map port 80 of the container to port 8080 of the host
   environment:
    WORDPRESS_DB_HOST: db:3306  # Database host
    WORDPRESS_DB_USER: wordpress
    WORDPRESS_DB_PASSWORD: wordpress
    WORDPRESS_DB_NAME: wordpress
depends_on:
```

```
      - db  # Ensures the db service starts first

  db:  # MySQL service
    image: mysql:5.7
    environment:
      MYSQL_ROOT_PASSWORD: rootpassword
      MYSQL_DATABASE: wordpress
      MYSQL_USER: wordpress
      MYSQL_PASSWORD: wordpress
```

How to Run It

To run a multi-container setup like the one above:

1. **Save the file as docker-compose.yml.**

   Or

   docker-compose.yaml

2. **To Start the compose**

   docker-compose up –d

3. **To stop the containers**

   docker-compose down

4. **To scale the container**

   docker-compose up --scale<service name>=2 -d

1.Define and run multiple interdependent services

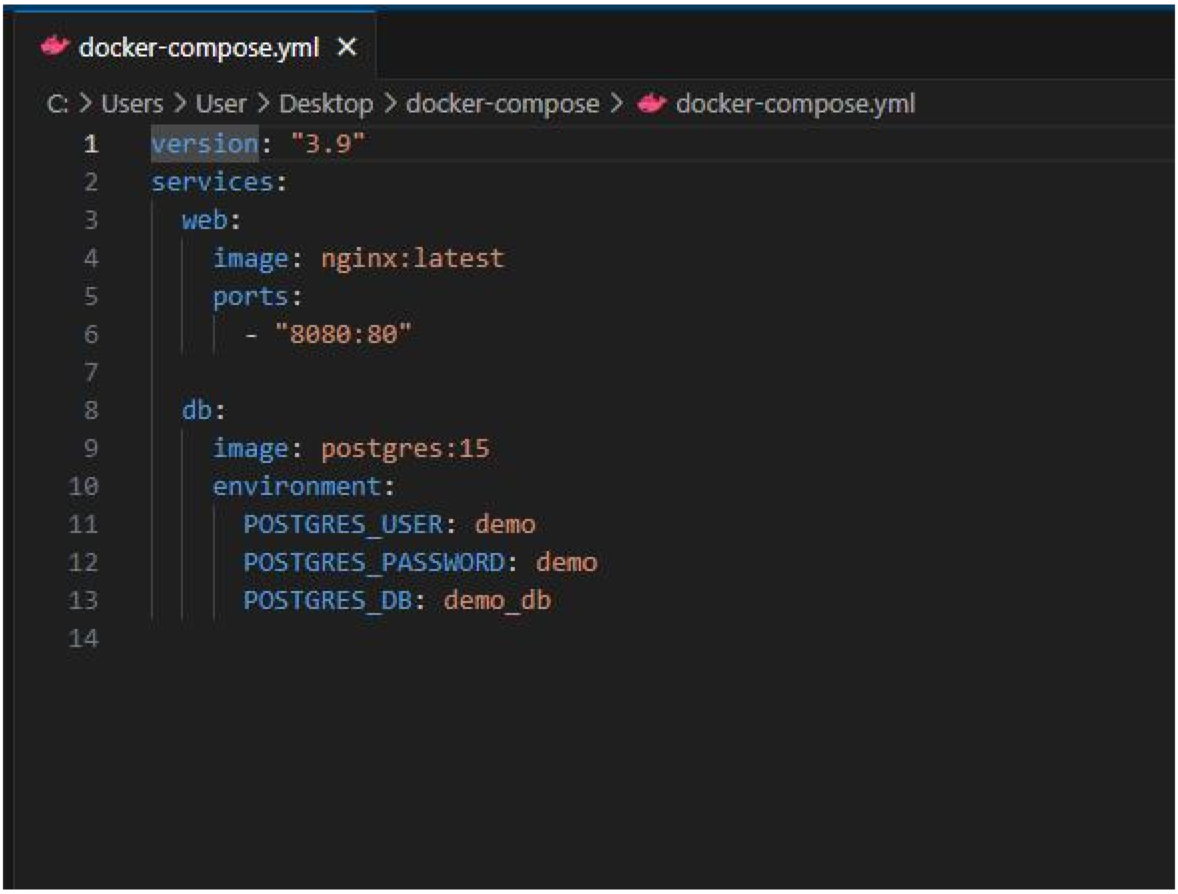
Task:

I. Create a new folder compose-lab

Inside it, create a file docker-compose.yml with the following content:

```
version: "3.9"
services:
  web:
```

```
    image: nginx:latest
    ports:
      - "8080:80"

  db:
    image: postgres:15
    environment:
      POSTGRES_USER: demo
      POSTGRES_PASSWORD: demo
      POSTGRES_DB: demo_db
```



II.Run the setup:

```
docker compose up -d
```

III.Open your browser and visit: http://localhost:8080.

IV.Expected Output:

Nginx welcome page is displayed.

db container runs in the background.

2.Write and interpret docker-compose.yml files

Task:

I. Modify docker-compose.yml to add a Redis cache:

```
redis:
   image: redis:alpine
```

II.Add a depends_on so web waits for Redis:

```
 web:
   image: nginx:latest
   ports:
     - "8080:80"
depends_on:
     - redis
```

```
docker-compose.yml ×
C: > Users > User > Desktop > docker-compose > docker-compose.yml
version: "3.9"
services:
  web:
    image: nginx:latest
    ports:
      - "8081:80"
    depends_on:
      - redis

  redis:
    image: redis:alpine

  db:
    image: postgres:15
    environment:
      POSTGRES_USER: demo
      POSTGRES_PASSWORD: demo
      POSTGRES_DB: demo_db
```

III.Restart the setup:

docker compose up –d

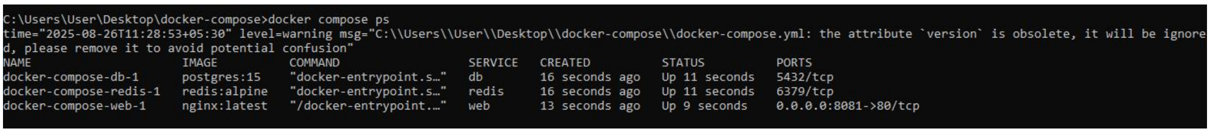
```
C:\Users\User\Desktop\docker-compose>docker compose up -d
time="2025-08-26T11:28:34+05:30" level=warning msg="C:\\Users\\User\\Desktop\\docker-compose\\docker-compose.yml: the attribute `version` is obsolete, it will be ignore
d, please remove it to avoid potential confusion"
[+] Running 4/4
 ✔ Network docker-compose_default    Created                                  2.1s
 ✔ Container docker-compose-db-1     Started                                  5.8s
 ✔ Container docker-compose-redis-1  Started                                  6.0s
 ✔ Container docker-compose-web-1    Started                                  4.0s
```

docker compose ps

```
C:\Users\User\Desktop\docker-compose>docker compose up -d
time="2025-08-26T11:25:22+05:30" level=warning msg="C:\\Users\\User\\Desktop\\docker-compose\\docker-compose.yml: the attribute `version` is obsolete, it will be ignore
d, please remove it to avoid potential confusion"
[+] Running 8/8
 ✔ redis Pulled                                                               12.6s
   ✔ 61cfb50eeff3 Download complete                                           1.2s
   ✔ 168694ef5d62 Download complete                                           1.2s
   ✔ 1f79dac8d2d4 Download complete                                           1.2s
   ✔ 4f4fb700ef54 Already exists                                              0.0s
   ✔ 9880d81ff87a Download complete                                           0.9s
   ✔ f8eab6d4856e Download complete                                           7.7s
   ✔ 9824c27679d3 Download complete                                           3.4s
[+] Running 3/3
 ✔ Container docker-compose-db-1     Running                                  0.0s
 ✔ Container docker-compose-redis-1  Started                                  3.6s
 ✔ Container docker-compose-web-1    Running                                  0.0s

C:\Users\User\Desktop\docker-compose>docker compose ps
time="2025-08-26T11:26:03+05:30" level=warning msg="C:\\Users\\User\\Desktop\\docker-compose\\docker-compose.yml: the attribute `version` is obsolete, it will be ignore
d, please remove it to avoid potential confusion"
NAME                     IMAGE          COMMAND                  SERVICE   CREATED          STATUS          PORTS
docker-compose-db-1      postgres:15    "docker-entrypoint.s…"   db        7 minutes ago    Up 7 minutes    5432/tcp
docker-compose-redis-1   redis:alpine   "docker-entrypoint.s…"   redis     28 seconds ago   Up 25 seconds   6379/tcp
docker-compose-web-1     nginx:latest   "/docker-entrypoint.…"   web       2 minutes ago    Up 2 minutes    0.0.0.0:8081->80/tcp
```

IV.Expected Output:

Three services (web, db, redis) are listed as running.

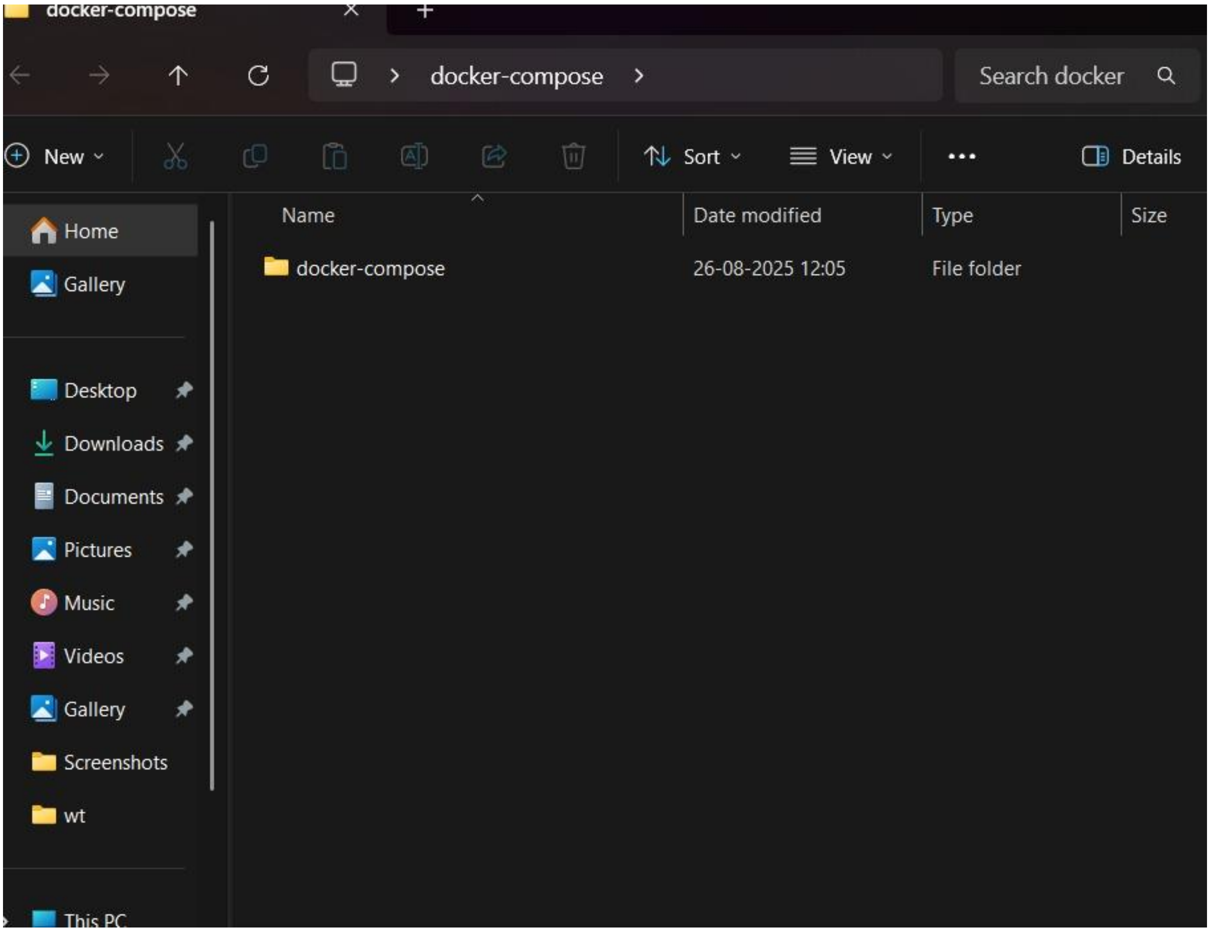


3.Deploy across different machines

Task:

I. Zip your compose-lab folder.

Transfer it to another machine with Docker Compose installed.



II.Run:

docker compose up -d

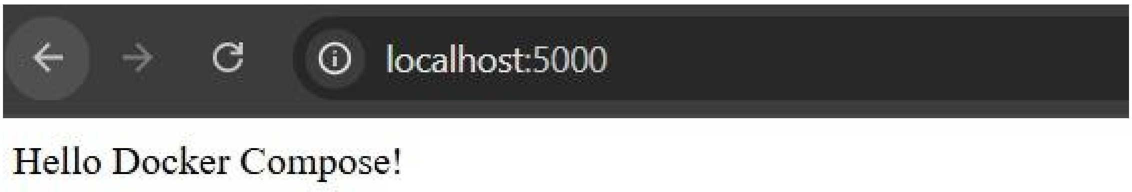
Check that Nginx and Postgres work there as well.

```
 ✔f5dfd246bc6e Pull complete                                                          97.0s
 ✔ba7036bb0a60 Pull complete                                                          97.0s
 ✔799f859abbbd Pull complete                                                          97.0s
 ✔d903e777080c Pull complete                                                          97.1s
 ✔04adcb462801 Pull complete                                                          97.1s
 ✔5c10925b29ff Pull complete                                                          97.1s
[+] Running 0/1.8s (4/8)                                                docker:desktop-linux
[+] Running 0/1 Building                                                              21.5s
[+] Building 27.4s (7/8)                                                docker:desktop-linux
[+] Building 28.2s (10/10) FINISHED                                     docker:desktop-linux
 => [web internal] load build definition from Dockerfile                                0.0s
 => => transferring dockerfile: 142B                                                    0.0s
 => [web internal] load metadata for docker.io/library/python:3.10-slim                 5.6s
 => [web internal] load .dockerignore                                                   0.0s
 => => transferring context: 2B                                                         0.0s
 => [web 1/4] FROM docker.io/library/python:3.10-slim@sha256:420fbb0e468d3eaf0f7e93ea6f7a48792cbcadc39d43ac95b96  16.2s
 => => resolve docker.io/library/python:3.10-slim@sha256:420fbb0e468d3eaf0f7e93ea6f7a48792cbcadc39d43ac95b96bee2a  0.0s
 => => sha256:7732878f45d9e71f91ce50493915297cca1bde392445d9ddcc0f378a200967bf 1.29MB / 1.29MB   1.6s
 => => sha256:72e8e193aa94d19c7f1bcbc00737a83d1906bcc1e51965c2873f081eb87bd3a0 14.10MB / 14.10MB 15.0s
 => => sha256:3a195ff1e16155a2ca71eee2cc2c4e467119c644d0360b7c2f6e6d9633f9358b 250B / 250B       1.7s
 => => sha256:420fbb0e468d3eaf0f7e93ea6f7a48792cbcadc39d43ac95b96bee2afe4367da 10.37kB / 10.37kB 0.0s
 => => sha256:e3c437eb1623e9086f42038c1273a7eff09cfff44d7c2ece721fa61560af7e1f 1.75kB / 1.75kB   0.0s
 => => sha256:c2c9cfb78b6afd7fa4f08488dabdd4482417fa3bbc3571425c20b53d66636fc0 5.38kB / 5.38kB   0.0s
 => => extracting sha256:7732878f45d9e71f91ce50493915297cca1bde392445d9ddcc0f378a200967bf       0.2s
 => => extracting sha256:72e8e193aa94d19c7f1bcbc00737a83d1906bcc1e51965c2873f081eb87bd3a0       1.1s
 => => extracting sha256:3a195ff1e16155a2ca71eee2cc2c4e467119c644d0360b7c2f6e6d9633f9358b       0.0s
 => [web internal] load build context                                                   0.0s
 => => transferring context: 221B                                                       0.0s
 => [web 2/4] WORKDIR /app                                                              0.2s
 => [web 3/4] COPY app.py /app/                                                         0.0s
 => [web 4/4] RUN pip install flask                                                     5.8s
 => [web] exporting to image                                                            0.2s
 => => exporting layers                                                                 0.1s
 => => writing image sha256:4fda6735cc10585ef46a42c8b6a2c9806942cbc441dbc207e5e3301422f92ebd  0.0s
[+] Running 6/6o docker.io/library/docker-compose-web                                   0.0s
 ✔Service web                      Built                                               28.5s
 ✔Network docker-compose_app-net   Created                                              0.1s
 ✔Network docker-compose_default   Created                                              0.0s
 ✔Volume "docker-compose_db-data"  Created                                              0.0s
 ✔Container docker-compose-db-1    Started                                              0.6s
 ✔Container docker-compose-web-1   Started                                              1.0s
```

III.Expected Output:

The same services run on the new machine without changes.

```
C:\Users\ranga\OneDrive\Desktop\docker-compose\docker-compose>docker compose ps
NAME                   IMAGE                COMMAND                   SERVICE   CREATED              STATUS               PORTS
docker-compose-db-1    postgres:15          "docker-entrypoint.s…"    db        About a minute ago   Up About a minute    5432/tcp
docker-compose-web-1   docker-compose-web   "python app.py"           web       About a minute ago   Up About a minute    0.0.0.0:5000->5000/tcp
```



4.Networking and persistent storage

Task:

I. **Update your docker-compose.yml to add a custom network and volume:**

```yaml
networks:
  app-net:

volumes:
db-data:

services:
  web:
    image: nginx:latest
    ports:
      - "8080:80"
    networks:
      - app-net
depends_on:
      - db

db:
    image: postgres:15
    environment:
      POSTGRES_USER: demo
      POSTGRES_PASSWORD: demo
      POSTGRES_DB: demo_db
    volumes:
      - db-data:/var/lib/postgresql/data
    networks:
      - app-net
```

docker-compose.yml

C: > Users > User > Desktop > docker-compose > docker-compose.yml

```yaml
services:
  web:
    image: nginx:latest
    ports:
      - "8081:80"
    networks:
      - app-net
    depends_on:
      - db

  db:
    image: postgres:15
    environment:
      POSTGRES_USER: demo
      POSTGRES_PASSWORD: demo
      POSTGRES_DB: demo_db
    volumes:
      - db-data:/var/lib/postgresql/data
    networks:
      - app-net

networks:
  app-net:

volumes:
  db-data:

```

II.Run:

docker compose up –d

```
[+] Running 0/1esktop\docker-compose>docker compose up -d
 - Network docker-compose_app-net  Creating                                                                0.3s
[+] Running 4/46T11:36:37+05:30" level=warning msg="Found orphan containers ([docker-compose-redis-1]) for this project. If you removed or renamed this service in your
 ✔ Network docker-compose_app-net    Created                                                               0.3s
 ✔ Volume "docker-compose_db-data"   Created                                                               0.0s
 ✔ Container docker-compose-web-1    Started                                                               9.7s
 ✔ Container docker-compose-db-1     Started                                                               3.1s
```

III.Insert some data into Postgres (optional with psql).

```
C:\Users\User\Desktop\docker-compose>docker compose exec db psql -U demo -d demo_db
psql (15.14 (Debian 15.14-1.pgdg13+1))
Type "help" for help.

demo_db=# CREATE TABLE test (id SERIAL PRIMARY KEY, name VARCHAR(50));
CREATE TABLE
demo_db=# INSERT INTO test (name) VALUES ('Sample data');
INSERT 0 1
demo_db=# SELECT * FROM test;
 id |    name
----+-------------
  1 | Sample data
(1 row)

demo_db=# \q
```

IV.Remove containers:

docker compose down

```
[+] Running 3/3esktop\docker-compose>docker compose down
 Container docker-compose-web-1  Removed
 Container docker-compose-db-1   Removed
 Network docker-compose_app-net  Removed
```

V.Start again:

docker compose up –d

```
C:\Users\User\Desktop\docker-compose>docker compose up -d
[+] Running 0/0
 - Network docker-compose_app-net  Creating                                                    0.0s
[+] Running 3/36T11:41:36+05:30" level=warning msg="Found orphan containers ([docker-compose-redis-1]) for this project. If you removed or renamed this service in your
 Network docker-compose_app-net  Created                                                       0.1s
 Container docker-compose-db-1   Started                                                       1.0s
 Container docker-compose-web-1  Started                                                       1.3s
```

VI.Expected Output:

Database data persists across restarts.

Services communicate via the app-net network using service names.

```
C:\Users\User\Desktop\docker-compose>docker compose ps
NAME                    IMAGE           COMMAND                   SERVICE   CREATED              STATUS               PORTS
docker-compose-db-1     postgres:15     "docker-entrypoint.s…"    db        About a minute ago   Up About a minute    5432/tcp
docker-compose-redis-1  redis:alpine    "docker-entrypoint.s…"    redis     14 minutes ago       Up 14 minutes        6379/tcp
docker-compose-web-1    nginx:latest    "/docker-entrypoint.…"    web       About a minute ago   Up About a minute    0.0.0.0:8081->80/tcp
```

5.Faster iteration during development

Task:

I. Create a simple Flask app in app.py:

from flask import Flask

app = Flask(__name__)

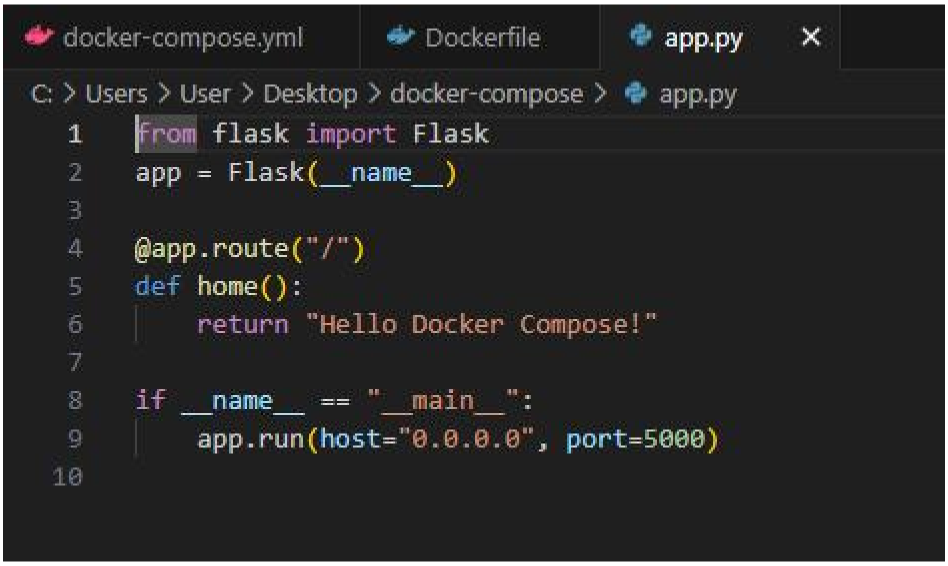
@app.route("/")

def home():

    return "Hello from Flask + Docker!"

if __name__ == "__main__":

app.run(host="0.0.0.0", port=5000)



II.Add a Dockerfile in the same folder:

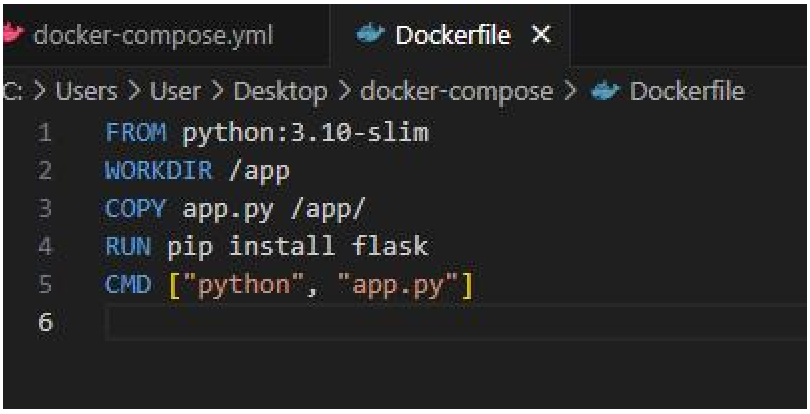
FROM python:3.10-slim

WORKDIR /app

COPY app.py /app/

RUN pip install flask

CMD ["python", "app.py"]



III.Update docker-compose.yml:

```
 web:
  build: .
  ports:
   - "5000:5000"
depends_on:
   - db
```

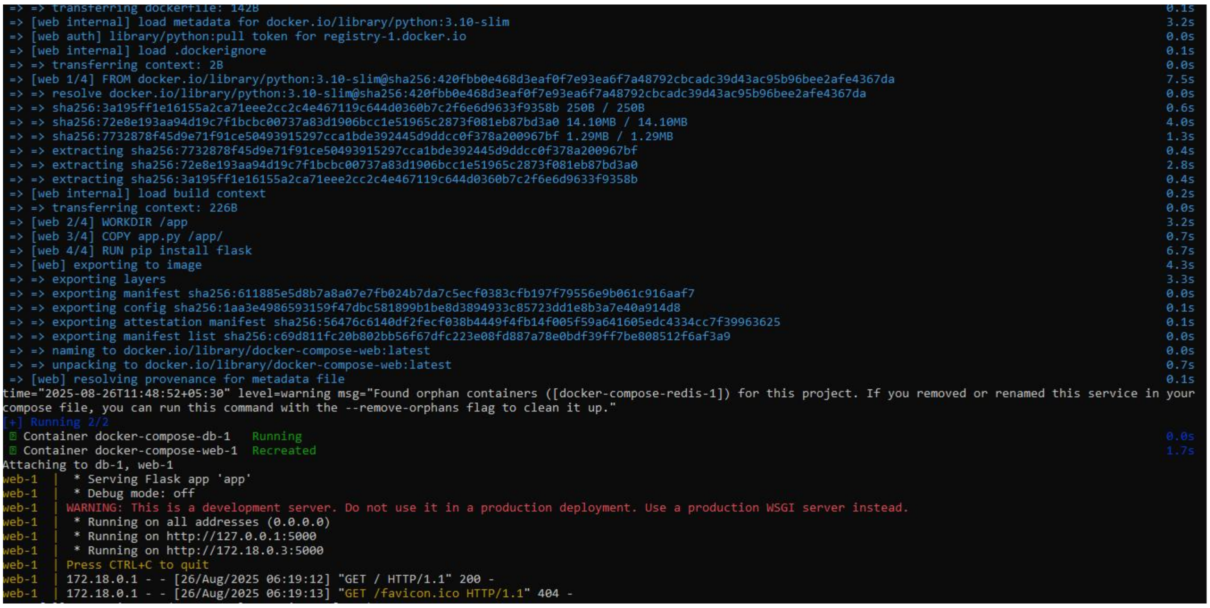
docker-compose.yml

C: > Users > User > Desktop > docker-compose > docker-compose.yml

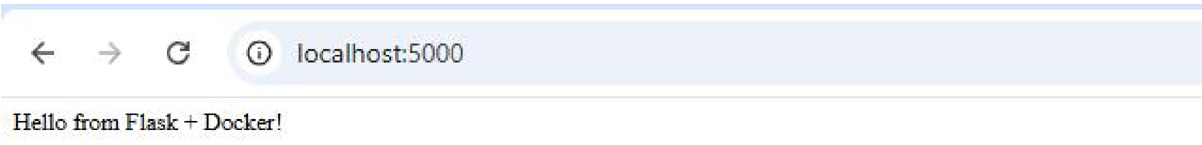
```yaml
services:
  web:
    build: .
    ports:
      - "5000:5000"
    depends_on:
      - db

  db:
    image: postgres:15
    environment:
      POSTGRES_USER: demo
      POSTGRES_PASSWORD: demo
      POSTGRES_DB: demo_db
    volumes:
      - db-data:/var/lib/postgresql/data
    networks:
      - app-net

networks:
  app-net:

volumes:
  db-data:
```

IV.Run:

docker compose up –build



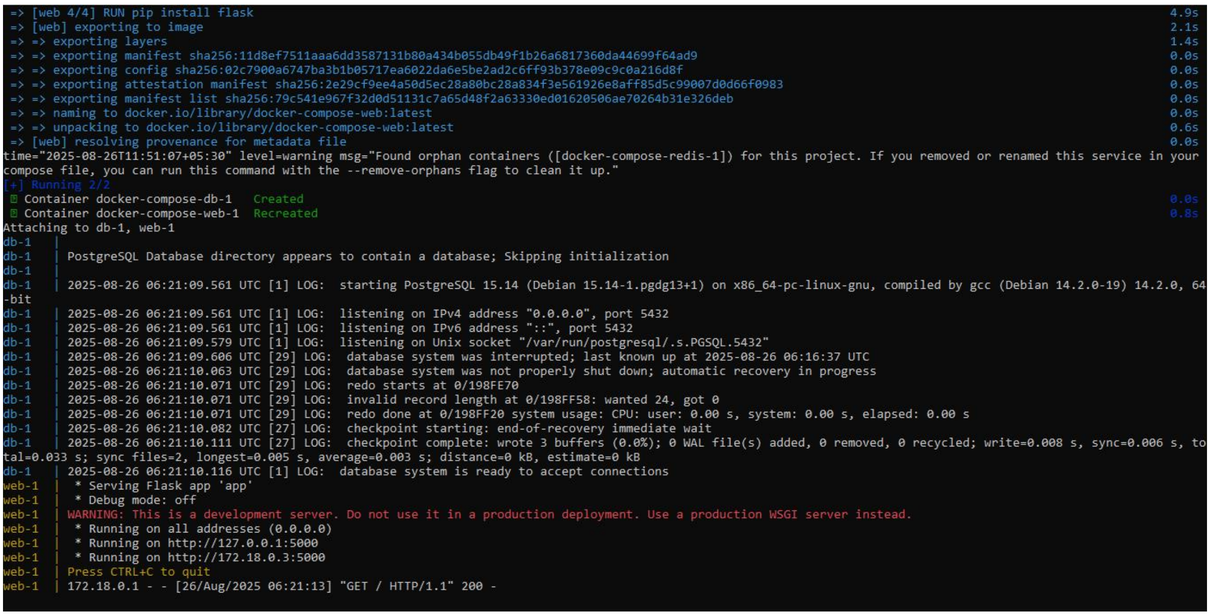
Visit http://localhost:5000.



Change the return text in app.py (e.g., "Hello Docker Compose!").

V.Rebuild:

docker compose up –build



VI.Expected Output:

New message appears instantly after rebuild.

Conclusion:

Docker Compose simplifies multi-container application management by allowing developers to define, configure, and run services in a single YAML file. It streamlines deployment, ensures consistent environments, and reduces manual setup, making it an essential tool for orchestrating complex, container-based applications efficiently.